Техническая документация

Платформа-агрегатор ИТ-вакансий с обратной механикой:
работодатель сам находит кандидата в категоризованной базе (специализация × подтверждённый тестом грейд)

Живой сервис: https://piratehunter.fun

OpenAPI: /docs на backend (BACK_ENV=local)

Репозиторий: публичный GitHub (см. README)

Дата сборки: 11 октября 2026

PirateHunter

Платформа подбора ИТ-специалистов с **обратной механикой**: работодатель
находит кандидата сам. Кандидаты автоматически категоризуются по
объективно подтверждаемому профилю — опрос по специализации, адаптивный
CAT-тест на заявленный грейд (грейд = результат теста, не стаж),
достижения ФСП — и попадают в категоризованную базу (7 специализаций ×
4 грейда). Работодатель описывает потребность, получает объяснимую
подборку («почему этот человек» — разложение score по факторам) и
направляет конкретному кандидату приглашение **с обязательной зарплатной
вилкой**. Контакты кандидата раскрываются только после принятия
приглашения. Достижения ФСП влияют на ранжирование внутри категории,
но не на грейд; кейс «нет истории ФСП» — первоклассный: профиль
показывается со статусом, участвует в подборке без буста.
Кейс ФСП (ЛЦТ 2026, спец. трек). **Живой сервис: https://piratehunter.fun**.
Репозиторий: https://github.com/piratehunter-team/piratehunter
(плейсхолдер — заменить на публичный к сдаче). Источник правды по
продукту — PRD.md; материалы кейса (ТЗ, брендбук) — в

materials/ (не в git). Техническая документация — docs/

(ARCHITECTURE, RUN, TESTING, VALIDATION, MATCHING, FSP-INTEGRATION, API;

единый PDF собирается docs/build_docs.py).

Стек: Python 3.13 · FastAPI · Pydantic v2 · SQLAlchemy 2 (async) ·

Alembic · PostgreSQL 16 ×2 · Redis 7 · numpy (IRT) · OpenRouter LLM

(Qwen-класс, с fallback) · Next.js 16 (App Router, FSD) · Bun ·

Tailwind v4 · zustand · Docker · Caddy · pytest/ruff/biome.

Быстрый старт

```
cp .env.example .env                          # 1. конфиг (дефолты для dev)
docker compose --env-file .env up -d --build  # 2. всё: 4 сервиса + PG×2 + Redis
./scripts/seed.sh                             # 3. эталонные демо-данные
```

Открыть http://localhost:56151. OpenAPI backend — http://localhost:64844/docs

(BACK_ENV=local). Сквозной смоук: ./scripts/smoke.sh (14 шагов по

живому стеку), сброс стенда: ./scripts/reset-dev.sh. Подробно —

docs/RUN.md.

Демо-аккаунты (dev-login)

На /login — кнопка входа без почты (BACK_AUTH_DEV_LOGIN=1, dev-дефолт;

эквивалент API: POST /api/v1/auth/dev {"account": "<имя>"}):

Аккаунт	Роль	Что показывает
candidate	Кандидат	Без истории ФСП (обязательный кейс «404»)
candidate-fsp	Кандидат	С привязанным ФСП ID fsp_001 — богатая история достижений
employer	Работодатель	Потребность → подборка с explanation → приглашение с вилкой

Вход через FSP ID: кнопка «Войти через FSP ID» на /login проходит

полный мок-OIDC флоу (fsp_001 → тот же демо-аккаунт candidate-fsp).

Экспресс-тест для жюри — 4 вопроса, 3–5 минут.

Архитектура

Монорепо, четыре сервиса, общение только по HTTP, ноль shared-кода. Персональные данные живут только в backend; core-api оперирует анонимными `candidate_id` (privacy by design). Internal-сервисы наружу не выставлены — с ними общается только backend внутри docker-сети.

Сервис	Порт	Назначение	Наружу
`frontend`	56151	Next.js (FSD): лендинг + ЛК кандидата + ЛК работодателя	да (через Caddy)
`backend`	64844	FastAPI `/api/v1`: auth (email-код, FSP ID, dev-login), профили, ПДн, приглашения	да (`/api/v1`)
`core-api`	51457	FastAPI internal: тест-движок (CAT-lite), матчинг с explanation, каталог	нет
`fsp-registry`	52457	FastAPI internal: мок-реестр достижений ФСП + мок-OIDC «FSP ID»	нет*
`pg-back` / `pg-core`	—	PostgreSQL 16, БД `ph_back` / `ph_core`	нет
`redis`	—	Redis 7: коды auth, OIDC state, кэш	нет

* В проде authorize-страница мок-OIDC доступна браузеру отдельным поддоменом `fsp-id.*` (см. `Caddyfile`). В dev порты опубликованы на `127.0.0.1` для отладки (у PG/Redis — 55432/55433/56379). Деплой на VPS (2 vCPU/4 ГБ): образы переносятся готовыми `docker save | ssh docker load` — docs/RUN.md.

Качество

202 автотеста (hunter_core 75 · backend 110 · fsp_registry 17), живой смоук 14 шагов, воспроизводимый цикл reset□smoke□seed, детерминированные сиды (302 кандидата / 16 компаний / 6 потребностей / 40 приглашений). Валидация (PRD §9): стабильность грейда 96.7% (порог 80%), ±1 — 100% (порог 97%), Spearman 0.972 (порог 0.85), Precision@5 0.85 (порог 0.8);

полный прогон 300×3: 99% / 0.977 — docs/VALIDATION.md.

Структура

```
piratehunter/
├── PRD.md              # источник правды по продукту
├── README.md
├── docker-compose.yml   # стек: 4 сервиса + PG×2 + Redis (+ back-migrate)
├── Caddyfile           # прод: TLS (Let's Encrypt), reverse proxy
├── .env.example
├── docs/               # техническая документация (ARCHITECTURE, RUN,
│                   #   TESTING, VALIDATION, MATCHING,
│                   #   FSP-INTEGRATION, API + build_docs.py → PDF)
├── backend/            # FastAPI · /api/v1 · ПДн · Alembic · pytest
├── hunter_core/        # core-api · engine/ (CAT, скоринг) · itembank/ ·
│                   #   validation/ (харнесс §9) · llm/
├── fsp_registry/       # мок-реестр достижений ФСП + мок-OIDC (stateless)
├── frontend/           # Next.js 16 FSD · айдентика ФСП (tokens.fsp.css)
├── scripts/            # reset-dev.sh · seed.sh · smoke.sh
└── materials/          # ТЗ, брендбук (не в git)
```

1. Архитектура

Архитектура PirateHunter

Платформа-агрегатор ИТ-вакансий с обратной механикой: работодатель сам находит кандидата в категоризованной базе. Четыре сервиса в монорепо, общение только по HTTP, ноль shared-кода (принципы унаследованы от проекта piratepitch). Живой сервис: **https://piratehunter.fun**.

Диаграмма

```
flowchart TD
  Browser[Браузер] -->|same-origin /api/* rewrites| FE[frontend :56151<br/>Next.js 16, FSD]
  Browser -->|OIDC authorize| FSP[fsp_registry :52457<br/>мок-OIDC «FSP ID»]
  FSP -->|303 code+state| FE
  FE -->|/api/v1 JWT| BE[backend :64844<br/>ПДн, auth, приглашения]
  BE -->|HTTP internal| CORE[core-api :51457<br/>CAT-тесты, матчинг]
  BE -->|HTTP internal| FSP
  BE --> PG1[(PG ph_back)]
  CORE --> PG2[(PG ph_core)]
  BE --> R[(Redis)]
  EXT[Caddy :80/443] --> FE
  EXT --> BE
```

Второй вариант — та же схема текстом (для печати):

```
            Браузер
      □□□□□□□□□□□□□□□□□□□□□□□□□□□□
      □ same-origin /api/*   □ OIDC: /oidc/authorize □ 303 (code,state)
      □                  □              □
frontend :56151      fsp_registry :52457 □□□□□□□□
Next.js (FSD)        мок-OIDC «FSP ID» + реестр достижений
      □                  □
      □ /api/v1 (JWT)       □ HTTP (внутр. docker-сеть)
      □                  □
backend :64844 □□□□□□□□□□□□□□□□
/api/v1 · auth · ПДн · приглашения
PG ph_back · Redis
      □ HTTP (внутр. docker-сеть, наружу не выставлен)
      □
core-api :51457 — CAT-тест-движок, матчинг, каталог
PG ph_core

Caddy :80/:443 □ frontend (сайт) + backend (/api/v1), TLS автоматически
```

Сервисы и порты

Сервис	Порт	Наружу	Назначение
frontend	56151	да (через Caddy)	Next.js 16 (App Router, FSD): лендинг + ЛК кандидата + ЛК работодателя одним приложением
backend	64844	да (`/api/v1`, JWT)	Auth (email-код / FSP ID / dev-login), профили, ПДн, потребности, приглашения, рынок вилок
core-api	51457	нет (internal)	Тест-движок CAT, матчинг, каталог специализаций, LLM-парсинг потребностей
fsp_registry	52457	нет (internal)*	Мок-реестр достижений ФСП + мок-OIDC-провайдер «FSP ID»
pg-back / pg-core	5432	нет	PostgreSQL 16, отдельная БД на сервис: `ph_back` / `ph_core`
redis	6379	нет	Коды auth, OIDC state, кэш профилей ФСП, лимиты
caddy	80/443	да	TLS (Let's Encrypt, HTTP-01), reverse proxy

* В проде fsp_registry выставляется отдельным поддоменом `fsp-id.*`

только для authorize-страницы мок-OIDC (браузер должен на неё зайти);

все остальные его эндпоинты — внутренние. См. `Caddyfile`.

Принципы (PRD §4.2)

- **Ноль shared-кода** — сервисы общаются только по HTTP. Frontend знает только backend; backend знает core-api и fsp_registry. Ни один сервис не импортирует код другого.
- **Свои PG + Redis** у backend и hunter_core — независимые релизы и миграции (Alembic у backend).
- **core-api и fsp_registry наружу не выставлены** — Caddy их не проксирует; в dev-compose порты опубликованы на `127.0.0.1` только для отладки и смоука.
- **ПДн только в backend** (privacy by design, аргумент для 152-ФЗ): ФИО, телефон, контакты, согласия живут в `ph_back` и никуда не уходят. core-api получает и хранит лишь анонимный `candidate_id` и оценочный снапшот. fsp_registry хранит синтетические достижения без ПДн; привязка `fsp_id` ↔ `candidate_id` — только в backend.
- **Детерминированные fallback везде**: LLM-фичи (парсинг потребности, перефразировка заданий) работают и без ключа/сети — демо не зависит от внешних API.

Потоки данных

Оценочный снапшот (backend → core-api)

После каждого изменения профиля (тест пройден, ФСП привязан/отвязан, профиль обновлён) backend пушит в core:

```
PUT /internal/snapshots/{candidate_id}
{candidate_id, spec, grade_verified, theta, test_score, fsp_score,
 stack[], updated_at}
```

`test_score: null` = «вычисли сам из θ». Матчинг читает **только** снапшоты — подборка не дёргает backend и не видит ПДн.

Результат подборки (core-api → backend)

`POST /internal/match` возвращает ranked `[{candidate_id, score, explanation, in_category}]` → backend джойнит с профилями (`ph_back`), применяет правила видимости (контакты скрыты до accept) и отдаёт фронту. Кандидаты вне категории идут отдельным блоком «рядом с запросом» (`in_category=false`).

ФСП (backend → fsp_registry)

Backend по `fsp_id` запрашивает `GET /fsp/v1/profiles/{fsp_id}`, считает `fsp_score` (формула — MATCHING.md), кэширует в Redis (TTL 3600 с) и включает в снапшот. **404 = нет истории** → `fsp_score = 0`, блок достижений не показывается (обязательный кейс ТЗ).

Тест (backend → core-api, проксирование)

Frontend ходит на `POST /api/v1/tests/sessions` и `POST /api/v1/tests/sessions/{id}/answers`; backend проксирует в `core-api /internal/tests/...` с подстановкой `candidate_id`, возвращает вопрос/результат. Состояние сессии CAT живёт в `ph_core`.

Вход через FSP ID (redirect-цепочка)

Backend `/api/v1/auth/fsp/authorize` отдаёт `authorize_url` на browser-facing адрес реестра (`BACK_FSP_PUBLIC_URL`, `login_hint` как у OIDC); реестр 303-ит браузер на `BACK_FSP_REDIRECT_URI` = GET-callback backend'а **на origin фронта** (Next проксирует `/api/*` — refresh-cookie `ph_refresh` ставится на домен фронта); backend обменивает code→token→userinfo и 303-ит на `/login/callback`, сессию поднимает bootstrap через `/auth/refresh`. Подробно — FSP-INTEGRATION.md.

Доменная модель (кратко, полная — PRD §5)

- **backend (`ph_back`)**: User, AuthCode, CandidateProfile (ФИО, контакты, стек, согласия 152-ФЗ), GradeState (кулдауны), Company,

Need (потребность), Vacancy, Invitation (обязательная вилка ЗП), Application. Инвариант ТЗ: `salary_from/to` NOT NULL, `from ≤ to`, `salary_type` обязателен (gross/net подписан в UI).

- **hunter_core (`ph_core`)**: ItemTemplate (параметризованный шаблон: генератор экземпляра по seed + детерминированный чекер), TestSession, Answer, AssessmentSnapshot (единственное, что читает матчинг), BetaStat (калибровка сложности).
- **fsp_registry**: stateless детерминированный генератор `FspProfile(fsp_id, rating, ranks, contests, events)`; БД не нужна.

Стек

Python 3.13 + FastAPI + Pydantic v2 + SQLAlchemy 2 (async) + Alembic, pytest, ruff; numpy для IRT-математики; LLM — OpenRouter (open-weights модель Qwen-класса) с детерминированным fallback. Frontend: Next.js 16.3.1 (App Router, React Compiler), Bun, Tailwind v4, Base UI, zustand, ky, biome, архитектура FSD (`src/fsd/*`). Айдентика ФСП — точные токены из брендбука (`accent #402FFF`, `ink #1B1C21`, Inter + JetBrains Mono), тема — сменяемый слой (`tokens.fsp.css`).

2. Запуск и развёртывание

Запуск и развёртывание PirateHunter

Живой сервис: **https://piratehunter.fun**. Инфраструктура: Docker + docker-compose + Caddy (авто-TLS). Вся система поднимается одной командой на чистой машине.

Локальный запуск (docker compose)

Требования: Docker с плагином compose, ~4 ГБ свободной RAM.

```
cp .env.example .env        # 1. секреты/конфиг (дефолты рабочие для dev)
docker compose --env-file .env up -d --build   # 2. всё: 4 сервиса + PG×2 + Redis
```

```
./scripts/seed.sh          # 3. эталонные демо-данные (302 кандидата, 16 компаний, ...)
```

Проверка health:

```
curl localhost:64844/health    # backend
curl localhost:51457/health    # core-api
curl localhost:52457/health    # fsp-registry
curl localhost:56151/          # frontend
curl localhost:52457/fsp/v1/profiles/fsp_001   # синтетический профиль ФСП (или 404)
```

OpenAPI backend — http://localhost:64844/docs (при `BACK_ENV=local`).

Вход по email-коду: без SMTP код пишется в лог backend

(`docker compose logs -f backend`, поле `dev_code`). При

`BACK_AUTH_DEV_LOGIN=1` (dev-дефолт) на `/login` — кнопка входа

демо-аккаунтами без почты: `candidate`, `candidate-fsp`, `employer`.

Порты в dev опубликованы на `127.0.0.1`: 56151 (frontend), 64844

(backend), 51457 (core-api), 52457 (fsp-registry), 55432/55433 (PG),

56379 (Redis) — для отладки и смоука.

Скрипты scripts/

Скрипт	Что делает
`./scripts/reset-dev.sh`	Стенд в чистое состояние: стоп backend/core, пересоздание схем `ph_back`/`ph_core` (DROP SCHEMA), подъём стека, дожди health. **Данные стираются.**
`./scripts/seed.sh`	Ресед эталона (PRD §10) одной командой: детерминированный seed `piratehunter-v1`, truncate доменных таблиц + очистка снапшотов в core. Перед защитой/после тестов жюри база возвращается к эталону.
`./scripts/smoke.sh`	Живой сквозной смоук по HTTP (14 шагов): health → dev-login → email-код → профиль → CAT-тест → потребность → shortlist → скрытость контактов → приглашение с вилкой → accept → OIDC-флоу FSP ID → LLM-парсинг → экспресс-тест → рынок вилок. Требует поднятый стек, `curl`, `jq`.

Воспроизводимый цикл: `reset-dev.sh` → `smoke.sh` (зелёный) → `seed.sh`.

Деплой на VPS (prod)

Целевой стенд: VPS 2 vCPU / 4 ГБ RAM, Ubuntu + Docker + compose-плагин.

В рантайме стек потребляет ~1.5 ГБ — VPS намеренно **не собирает

образы** (сборка Next.js и pip/uv-зависимостей не влезла бы в RAM и

заняла бы десятки минут): образы собираются на домашней машине и

переносятся готовыми.

Подготовка (один раз)

1. **DNS:** A-записи `piratehunter.fun` (и `www.`) → IP VPS. Для

мок-OIDC — поддомен `fsp-id.piratehunter.fun` → тот же IP.

1. **На VPS:**

```bash

ufw allow 80,443/tcp && ufw enable # Caddy выпускает TLS (HTTP-01)

# ssh-ключ, docker, compose-плагин

git clone <repo> piratehunter && cd piratehunter

cp .env.example .env # прод-значения (см. ниже)

```

Прод-.env (отличия от dev)

```
BACK_ENV=prod               # /docs закрыт
BACK_AUTH_DEV_LOGIN=0       # dev-login выключен
NEXT_PUBLIC_DEV_LOGIN=0
BACK_JWT_SECRET=<длинная случайная строка>
BACK_CORS_ORIGINS=https://piratehunter.fun
BACK_FSP_PUBLIC_URL=https://fsp-id.piratehunter.fun
BACK_FSP_REDIRECT_URI=https://piratehunter.fun/api/v1/auth/fsp/callback
BACK_FRONT_URL=https://piratehunter.fun
# SMTP (коды на почту): BACK_SMTP_HOST/PORT/USER/PASSWORD/FROM
```

В `Caddyfile` заменить `piratehunter.example.com` → `piratehunter.fun`

и раскомментировать блок `fsp-id.*` (мок-OIDC наружу). Важно:

Caddyfile примонтирован файлом — после правок

`docker compose … up -d --force-recreate caddy`.

Перенос образов и старт

```
# На домашней машине (сборка):
docker compose --env-file .env build
docker save $(docker compose --env-file .env config --images) | \
  ssh vps 'docker load'
```

```
# На VPS:
docker compose --env-file .env up -d     # без --build: образы уже загружены
docker compose ps                        # все healthy
./scripts/seed.sh                        # эталонные демо-данные
```

Caddy сам выпускает и продлевает сертификаты Let's Encrypt (HTTP-01, порты 80/443), редиректит HTTP→HTTPS, проксирует сайт на frontend, а `/api/v1` — на backend. core-api наружу не торчит (PRD §4.2).

Ресед эталона одной командой

```
./scripts/seed.sh    # truncate + детерминированная заливка (seed piratehunter-v1)
```

Сводка эталона: 302 кандидата (подтверждённый грейд — 229 ✓ 76%, с историей ФСП — 181 ✓ 60%), 16 компаний, 6 потребностей, 40 приглашений, в ранжировании категории backend×middle — 19 кандидатов.

Тестирование развёртывания

Смоук (`scripts/smoke.sh`) бьёт по `localhost`-портам; на проде достаточно ручного прохода сквозного сценария из PRD §1 на `https://piratehunter.fun` + health-чеков. Подробнее — TESTING.md.

3. Тестирование

Тестирование PirateHunter

Покрытие трёх слоёв: (1) юнит- и API-тесты каждого сервиса на pytest, (2) живой сквозной смоук `scripts/smoke.sh` по HTTP по поднятому docker-стеку, (3) браузерные E2E-проверки фронта. Плюс воспроизводимый цикл «reset ✓ smoke ✓ seed» и детерминированные сиды.

1. Автотесты (pytest)

Цифры — фактические прогоны `pytest -q` в каждом сервисе (11.10.2026):

Сервис	Тестов	Как гоняются	Что покрывают
hunter_core	**75**	чистый pytest, без сети и БД	CAT-движок (θ-обновление, стоп-правила, исходы confirmed/upgrade/downgrade), скоринг и explanation, fsp_score, стек-матчинг, свежесть, каталог, фабрики шаблонов (генерация экземпляров + чекеры), валидационный харнесс
backend	**110**	pytest + aiosqlite, core/fsp клиенты мокаются без сети	auth (email-код, dev-login, refresh-cookie), роли и зависимости, профили, правила видимости контактов (скрыто до accept), приглашения (вилка обязательна — 422 без неё), снапшоты в core, FSP-привязка (включая 404 «нет истории»), кулдауны грейда (409), каталог, рынок вилок, резюме
fsp_registry	**17**	чистый pytest	детерминированный генератор профилей (~60/40 история/404), мок-OIDC (authorize → code → token → userinfo, одноразовость кода, TTL, невалидный client/redirect)

Итого **202 теста**, все зелёные. Запуск:

```
cd backend      && uv run pytest -q && uv run ruff check .
cd hunter_core  && uv run pytest -q && uv run ruff check .
cd fsp_registry && uv run pytest -q && uv run ruff check .
```

Фронт: `cd frontend && bun run lint && bun run build` (biome + tsc + прод-сборка Next.js). Кейс «404 = нет истории ФСП» в живом контуре проверяется вручную демо-аккаунтом `candidate` (см. FSP-INTEGRATION.md).

Точечные приёмы: backend гоняется на aiosqlite (паттерн piratepitch) — миграции и репозиторный слой без поднятия PG; core/fsp клиенты подменяются фейками, внешних вызовов в тестах нет.

2. Живой сквозной смоук — scripts/smoke.sh

14 шагов по реальному стеку (`docker compose up`), каждый — HTTP-вызов с проверкой результата (`curl` + `jq`), стоп по первому FAIL:

1. **Health всех сервисов** — backend/core-api/fsp-registry `/health` + frontend `/` и `/login`.
2. **Dev-login** — демо-аккаунты `candidate-fsp` (кандидат с ФСП) и `employer`, получение JWT.

3. **Email-код** — auth/email/request → dev_code (при BACK_ENV=local без SMTP) → auth/email/verify → токен.
4. **Профиль кандидата** — PUT /candidate/profile (стек, специализация, заявленный грейд, контакты).
5. **CAT-тест через backend → core-api** — старт сессии на middle, цикл ответов до результата {theta, grade_verified, outcome}.
6. **Компания + потребность** — PUT /employer/company, POST /employer/needs (backend проксирует LLM-парсинг «мы поняли так»).
7. **Shortlist** — GET /employer/needs/{id}/shortlist: непустая подборка, у каждой строки есть explanation.
8. **Скрытость контактов** — наш кандидат в банке, contacts: null до accept (privacy-правило §7.6).
9. **Приглашение** — без вилки → **422**; с вилкой 250–350k gross → 201 (инвариант ТЗ).
10. **Accept** — кандидат: до accept контакты компании скрыты, после accept раскрываются **обеим** сторонам (кандидат видит HR-контакты, работодатель — email кандидата).
11. **Полный OIDC-флоу «FSP ID»** — redirect-цепочка: authorize_url → 303 реестра (code+state) → GET-callback на origin фронта → 303 на /login/callback с refresh-cookie ph_refresh → /auth/refresh → access; users/me подтверждает fsp_id=fsp_001.
12. **LLM-парсинг потребности** — needs/parse распознаёт «мидла на FastAPI, PG, Redis» → spec=backend, grade=middle (работает и без ключа — детерминированный fallback).
13. **Экспресс-режим теста (для жюри)** — mode=express завершается → 4 вопросов, mode=express в результате.
14. **Рынок вилок** — market/salary?spec=backend&grade=middle: медиана по потребностям и приглашениям платформы (p50 from=250000, to=350000 на эталонном сиде).

Финал: «СМОУК ПРОЙДЕН — все 14 шагов зелёные».

3. Воспроизводимость

```
./scripts/reset-dev.sh   # чистые схемы ph_back/ph_core, стек переподнят, health зелёный
./scripts/smoke.sh       # 14 шагов — зелёные
./scripts/seed.sh        # эталон piratehunter-v1: 302 кандидата / 16 компаний / 6 потребностей / 40 приглашений
```

Сиды детерминированы (seed piratehunter-v1), датасет воспроизводим побайтово — перед защитой и после тестов жюри база возвращается к эталонному состоянию одной командой.

4. E2E браузерные проверки фронта

Фронт (Next.js 16, FSD) проверяется в браузере по сквозному сценарию PRD §1: лендинг → логин (email-код / «Войти через FSP ID» / dev-кнопка)

 онбординг кандидата адаптивный тест с таймером экран категории ЛК работодателя: потребность «мы поняли так» подборка с панелью «Почему в подборке» приглашение с вилкой инбокс кандидата с вилкой крупно accept раскрытие контактов. Проверяются money-экраны PRD §7.11 (подборка с explanation и виджетом «вилка vs рынок», тест, приглашение, категория). Кабинеты фазы 1 на моках (`NEXT_PUBLIC_API_MOCK=1`), auth — всегда живая.

Что осознанно не автоматизировано

Песочница исполнения code-заданий и прокторинг — плюс, не MVP (Q&A 06.10); code-шаблоны заложены в модель, при нехватке времени выкидываются без потери баллов. Нагрузочное тестирование не требуется оценкой (десятки пользователей, подборка < 2 с на 300+ кандидатах по снапшотам).

4. Валидация

Валидация решения (PRD §9)

Собственная процедура валидации (ТЗ §2, критерий «Тестирование + валидация + категоризация», 25%). Эталонных датасетов нет — построен синтетический контур: синтетические агенты-кандидаты с известным истинным уровнем, ручная разметка набора «потребность × кандидаты». Харнесс — CLI в `hunter_core/validation/`, артефакты прогона — `hunter_core/out/validation/`.

Методология

Синтетические агенты (§9.1). Агент с истинным уровнем `θ_true` отвечает верно с вероятностью `P = σ(θ_true  β)` по логистической модели IRT (β — сложность шаблона), плюс шум по времени ответа. Агенты равномерно покрывают шкалу θ. Контрольный прогон: **60 агентов × 2

повторных теста**, режим full (10 вопросов), специализация backend,

банк [28, 28] шаблонов по повторам, seed 42. Дополнительно выполнен

полный прогон **300 агентов × 3 повтора** (итоги ниже).

Метрики (§9.2).

1. Стабильность: повторные тесты агента % точного совпадения грейда, % в пределах ±1 грейда.
2. Дискриминативность: монотонность медиан присвоенного θ по полосам θ_true; Spearman(θ_true, θ_est); confusion-матрица; путаница соседних грейдов.
3. Эквивалентность экземпляров: для каждого шаблона разброс доли верных ответов по 50 seeds — доля seeds вне 95% биномиального ДИ (проверка, что параметризация по seed не меняет сложность).
4. Калибровка β: сходимость BetaStat (метод моментов по доле верных с поправкой на θ ответивших, демпфирование) к эмпирической сложности.
5. Релевантность подбора: ручная разметка 20 пар «потребность × 50 кандидатов» Precision@5, nDCG@10.
6. Пересечение наборов заданий между сессиями (антислив, Jaccard).

Пороги приёмки (§9.3) — зафиксированы до прогона

Метрика	Порог
Стабильность: точное совпадение грейда	80%
Стабильность: в пределах ±1 грейда	97%
Spearman(θ_true, θ_est)	0.85
Путаница соседних грейдов	< 15%
Доля seeds вне ДИ (эквивалентность экземпляров)	15%
Ошибка калибровки β	0.3
Precision@5 на размеченном наборе	0.8

Результаты контрольного прогона (60 агентов × 2 повтора)

Источник: `hunter_core/out/validation/summary.txt`,

validation.json, matching.json.

Метрика	Факт	Порог	Вердикт
Стабильность грейда, точное совпадение	**96.7%**	≥ 80%	PASS
Стабильность, в пределах ±1	**100.0%**	≥ 97%	PASS
Spearman(θ_true, θ_est)	**0.9718**	≥ 0.85	PASS
Путаница соседних грейдов	**1.7%**	< 15%	PASS
Доля seeds вне 95% ДИ	**4.2%** (битых экземпляров: 0)	≤ 15%	PASS
Ошибка калибровки β (BetaStat)	**0.125** (β 1.2 → 0.474 при эмпирической 0.349)	≤ 0.3	PASS
Precision@5 (20 потребностей × 50 кандидатов)	**0.85**	≥ 0.8	PASS
nDCG@10	0.9446	—	—
Пересечение наборов шаблонов (Jaccard, 1770 пар сессий)	0.305	—	низкое — антислив работает

Медианы присвоенного θ_est по полосам θ_true **монотонны**:

intern −2.233 < junior −0.109 < middle 1.226 < senior 2.973.

Confusion-матрица (строки — истинный грейд, столбцы — присвоенный;

точность строк: intern 100.0% (n=30), junior 96.7% (n=30), middle 96.7%

(n=30), senior 100.0% (n=30)):

```
          присвоенный
        intern  junior  middle  senior
intern      30       0       0       0
junior       0      29       1       0
middle       0       1      29       0
senior       0       0       0      30
```

Графики прогона:

- confusion.png — confusion-матрица «истинный × присвоенный» грейд;
- theta_scatter.png — рассеяние θ_est от θ_true с полосами грейдов.

Эквивалентность экземпляров проверена на 10 шаблонах × 50 seeds × 100 агентов (θ = 1.0): средняя доля seeds вне ДИ 4.2%, битых экземпляров (генератор/чекер не сошлись) — 0. Правило релевантности в подборе: «spec совпал И grade требуемого И stack_overlap 0.5» — разметка команды, набор и результаты по потребностям — в `matching.json` (per-need Precision@5 0.6–1.0, overall_pass=true).

Полный прогон (300 агентов × 3 повтора)

По итогам полного прогона (PRD §14, фаза 0): стабильность грейда **99%** (точное совпадение), Spearman **0.977** — пороги §9.3 выполнены с запасом.

Итог

ИТОГ: PASS — все пороги §9.3 выполнены, шесть из шести валидационных проверок зелёные, плюс метрики подбора. Пороги грейдов (intern < 1 junior < 0.5 middle < 2 senior) калиброваны этими прогонами; жюри может само пройти пилот экспресс-режимом (4 вопроса, 3–5 минут) и сверить присвоение грейда.

5. Механика подбора

Механика подбора (PRD §7.5)

Критерий «Механика подбора» — 35% оценки. Подбор — внутри core-api, работает только по оценочным снапшотам (privacy by design, PRD §4.3); backend применяет фильтры до скоринга и правила видимости после.

Формула скоринга

```
score = 0.40·test_score + 0.30·stack_match + 0.20·fsp_score + 0.10·freshness
```

Реализация: `hunter_core/engine/scoring.py` (`match_score`), веса — `DEFAULT_WEIGHTS` (конфигурируемы).

Обоснование порядка весов. Подтверждённая тестом компетентность (0.40) важнее самоописанного стека (0.30): грейд присваивается только адаптивным тестом (Q&A 07.10), стек — декларация кандидата. Достижения ФСП (0.20) — верифицированный внешний сигнал, усиливают ранжирование **внутри** категории, но не влияют на грейд (обязательное правило Q&A). Свежесть (0.10) — мягкий буст активных профилей, чтобы выдача не застаивалась.

Факторы

test_score (0.40)

Позиция θ внутри θ-полосы грейда: `(θ − lo) / (hi − lo)`, кламп 0..1.

Полосы: intern [−4, −1), junior [−1, 0.5), middle [0.5, 2), senior [2, 4]. В снапшоте может прийти готовое значение (`test_score: null` = вычисли из θ).

Unverified-потолок 0.4. Кандидат без подтверждённого грейда показывается **со статусом «не подтверждён»** и опускается в выдаче (потолок test_score = 0.4), но **не скрывается** — иначе выпадает из оборота (Q&A 06.10). Реализация — `UNVERIFIED_CAP = 0.4`, explanation помечает «(не подтверждён — потолок 0.4)».

stack_match (0.30)

`Σ w(совпавший навык)`: совпадение по must весит ×3, nice ×1; нормировка `3·|must| + |nice|` к 0..1; регистронезависимо; пересечение must∩nice считается must. Пустые требования → 0.0 (нет сигнала, а не штраф). Совпадение — точное по тегам стека снапшота.

fsp_score (0.20)

Считается backend'ом по достижениям ФСП (PRD §7.9, критерии — по Q&A 06.10/07.10), в core приходит готовым в снапшоте:

Достижение	Вес
Победитель (1 место)	1.0
Диплом I–III степени	0.7
Призовое место (4–10)	0.5
Участие (11+)	0.15
Мероприятие (роль/эксперт)	0.05

- **Дисциплины и уровни соревнований не ранжируются** — «все соревнования равно важны» (Q&A 06.10). Считаем результативность и число.
- **Антифарм-затухание явками**: i-е участие даёт `×0.5^(i−1)` к весу участия (0.15, 0.075, 0.0375, …), мероприятия — аналогично. Защита от набивания профиля одними онлайн-явками. Результативные достижения (1.0/0.7/0.5) затуханию не подвержены.
- **Cap 1.0.** Нет истории ФСП → 0, штрафа сверх веса нет.
- Командные результаты учитываются как обычные достижения (вклад честно не выделить — Q&A). Давность не штрафуем (архива на 2–4 года нет).
- Опционально post-MVP (заложено в модель): небольшой бонус за релевантность дисциплины специализации кандидата.

freshness (0.10)

Экспоненциальное затухание от последней активности (тест / задание / обновление профиля): `0.5^(days / 30)` — полураспад 30 дней. Нет данных об активности → 0.0.

Конвейер подборки (`POST /internal/match`)

1. Фильтры работодателя (spec, grade, стек, «есть ФСП») — **до** скоринга (PRD §7.5.2).
1. Снапшоты специализации сплитятся:

- **Категория** (in_category=true): grade_verified есть и

 требуемому грейду. Скоринг полной формулой, сортировка по score.

- **«Рядом с запросом»** (in_category=false): грейд ниже

требуемого или не подтверждён — те же снапшоты, тот же score, но

отдельным блоком после категории. Категорийность не ломается,

находчивость добавляется.

1. core + near, top-limit (default 20), детерминированный тай-брейк

по candidate_id.

1. Backend джойнит профили, применяет видимость контактов (только после

accept) и отдаёт фронту.

Ранжирование внутри категории (§7.5.4)

GET /internal/category/{spec}/{grade}/ranking — тот же score в режиме

просмотра категории **без потребности**: stack-фактор заменяется

нейтральной константой NEUTRAL_STACK_VALUE = 0.5 (его вклад 0.30·0.5

одинаков для всех — не влияет на порядок, честно отражён в explanation:

«стек: нейтральная константа»). Выше оказываются сильнее подтверждённые

(θ в полосе) профили с достижениями ФСП — формулировка ТЗ §2.

Объяснимость (§7.5.3) — обязательный артефакт

Каждая строка выдачи несёт explanation — разложение по 4 факторам в

порядке весов: {factor, weight, value, contribution, human_text}.

Примеры human_text:

- «тест: грейд middle, θ=1.20 0.47»
- «стек: совпало must 4/5, nice 2/3 0.76»
- «достижения ФСП: суммарный счёт 0.72»
- «свежесть профиля: активность 3 дн. назад 0.93»

Фронт рендерит панель «Почему в подборке» — прямое попадание в критерий

«обоснованность ранжирования и объяснимость выдачи».

Рыночная аналитика вилок (§7.5.5)

Раз ЗП обязательна и прозрачна (ТЗ §2.2.1), принцип превращён в фичу:

`GET /api/v1/employer/market/salary?spec&grade` — виджет «вилка vs

рынок по категории». Агрегат **по данным платформы** (анонимно):

перцентили p25/p50/p75 границ вилок потребностей (`sample_needs`) и

приглашений (`sample_invitations`) категории spec×grade + размер

категории из core (`category_size`). Пустая выборка перцентили null,

1. Работодатель видит «ваша вилка ниже медианы backend×middle» до

отправки приглашения. Стоимость реализации — один SQL-агрегат.

Схема «потребность → подборка → приглашение»

1. Работодатель описывает потребность свободным текстом

LLM-парсинг `POST /internal/llm/parse-need` (`{spec, grade,

stack_must, stack_nice, confidence, notes}`; детерминированный

keyword-fallback без ключа; невалидные slug'и null) экран «мы

поняли так» с правкой подтверждённая структура идёт в матчинг.

1. Shortlist: match по снапшотам ranked + explanation + блок

«рядом с запросом».

1. Приглашение конкретному кандидату: текст + **обязательная вилка

ЗП** (salary_from/to + salary_type gross/net) + способ связи;

вакансия опциональна (ТЗ §2.2). Без вилки — 422.

1. Кандидат видит вилку до ответа; статусы

`sent  viewed  accepted | declined`; после accept контакты

раскрываются обеим сторонам.

6. Интеграция с ФСП

Интеграция с ФСП (мок-реестр + мок-OIDC «FSP ID»)

Сервис `fsp_registry` играет двойную роль (Q&A 06.10): **мок-реестр достижений участника ФСП** (контракт `GET /fsp/v1/profiles/{fsp_id}`) и **мок-провайдер входа «FSP ID»** (имитация OAuth/OIDC-флоу). После хакатона мок заменяется боевым реестром/Keycloak — backend не меняется, меняется только конфиг. Это «схема предполагаемой интеграции» из ТЗ §3.4, исполненная в коде.

Контракт реестра достижений

```
GET /fsp/v1/profiles/{fsp_id}
→ 200 {fsp_id, rating, ranks[{discipline, rank, date}],
    contests[{name, discipline, date, place, diploma 1|2|3|null,
        team, team_result{team_name, place}}],
    events[{name, date, role}]}
→ 404 — истории ФСП нет
```

Детерминированный генератор (`fsp_registry/app/profiles/service.py`): seed = `sha256(соль + fsp_id)` — один и тот же `fsp_id` всегда даёт один и тот же профиль, в том числе между перезапусками. По хэшу **~60% fsp_id имеют историю, ~40% → 404** (пользователь без истории — обязательный кейс ТЗ). Распределение результатов: ~20% победители с дипломом I–III, ~25% призовые места 4–10, остальные участники; ~35% командных; рейтинг, разряды (0–3 дисциплины), мероприятия (0–4). Поле `level` (regional/national) сознательно не заводим — эксперты против ранжирования дисциплин и уровней (§7.9). Сервис stateless, БД не нужна. ПДн не хранит — «человекочитаемость» обеспечивает backend (у себя маппинг `fsp_id → candidate_id`).

Мок-OIDC flow входа «FSP ID»

Контракт приближен к OIDC, чтобы переезд на боевой Keycloak был

конфигом. Эндпоинты fsp_registry:

Метод	Путь	Назначение
GET	/oidc/authorize?client_id&redirect_uri&state&login_hint	авторизация: автовыбор демо-личности или явный fsp_id; 303 на redirect_uri с code+state
POST	/oidc/token (form)	обмен кода: {access_token, id_token, expires_in} — self-contained JWT HS256
GET	/oidc/userinfo (Bearer)	{sub=fsp_id, name, email}
GET	/.well-known/openid-configuration	discovery (issuer = browser-facing адрес)

Коды одноразовые, in-memory с TTL; client_id валидируется,

redirect_uri — по allowlist; обмен привязан к паре code+redirect_uri

+client_id (повторный обмен invalid_grant).

Redirect-цепочка (как это проходит в браузере)

1. Фронт: GET /api/v1/auth/fsp/authorize?fsp_id=... backend отдаёт

{authorize_url} — browser-facing адрес реестра

(BACK_FSP_PUBLIC_URL), OIDC-параметрами (client_id,

redirect_uri, state, login_hint = fsp_id).

1. Браузер идёт на /oidc/authorize реестр **303**-ит на

BACK_FSP_REDIRECT_URI = **GET-callback backend'а на origin фронта**

(https://piratehunter.fun/api/v1/auth/fsp/callback?code&state) —

Next проксирует /api/*, поэтому refresh-cookie ph_refresh

ставится на домен фронта.

1. Backend валидирует одноразовый state (Redis, TTL 600 с), обменивает

code token userinfo у реестра, создаёт/находит User по

fsp_id, подтягивает достижения, **303**-ит браузер на

/login/callback.

1. Сессия поднимается bootstrap-ом: /api/v1/auth/refresh по

ph_refresh-cookie access в память (zustand), single-flight

refresh в ky.

1. POST-вариант callback (code+state в теле) оставлен для API-клиентов.

Сквозной флоу покрыт смоуком (шаг 11): authorize_url 303 (code+state)

 callback 303 `/login/callback` cookie refresh `users/me` с

`fsp_id=fsp_001`.

Кейс «404 = нет истории ФСП» (обязательный по ТЗ)

- Привязка ФСП ID (автоматически при входе через FSP ID или вручную в

настройках): `fsp_registry` вернул 404 `fsp_score = 0`, блок

достижений **не рендерится**, UI подсказывает «привяжите ФСП ID».

- Кандидат **полностью не скрывается**: показывается со статусом,

участвует в подборке с unverified-потолком test_score (0.4), если

грейд подтверждён — в категории на общих основаниях.

- Фильтр работодателя «есть достижения ФСП» корректно исключает таких

кандидатов.

- Путь покрыт тестами backend (FSP-привязка с 404, «404 = нет истории

score 0, блок отсутствует»); ручная демонстрация — демо-аккаунт

`candidate` (без ФСП) против `candidate-fsp` (fsp_001 — богатая

история). Достижения кэшируются в Redis (TTL 3600 с).

Критерии влияния достижений (Q&A 06.10 / 07.10)

- Достижения ФСП — **буст внутри грейда** (фактор fsp_score, вес 0.20),

на грейд не влияют: грейд присваивается только тестом.

- Дисциплины равны, уровни соревнований не ранжируются.
- Антифарм: затухание повторных явок `×0.5^(i1)`; cap 1.0.
- Командный результат — как обычное достижение; давность не штрафуем.
- Формула и веса — в MATCHING.md; бонус за релевантность дисциплины

специализации — опционально post-MVP (модель `discipline` заложена).

Замена мока на боевой Keycloak — конфигом

Backend ходит к провайдеру через интерфейс `IdentityProvider`

(провайдеры `fsp-id` и `email-code`); клиент мок-OIDC параметризован:

```
BACK_FSP_PUBLIC_URL=https://fsp-id.piratehunter.fun   # issuer/authorize
BACK_FSP_REGISTRY_URL=http://fsp-registry:52457       # внутри сети
BACK_FSP_REDIRECT_URI=https://piratehunter.fun/api/v1/auth/fsp/callback
BACK_FSP_CLIENT_ID / BACK_FSP_CLIENT_SECRET           # client-конфиг
```

Переезд: поднять Keycloak с realm ФСП, завести client

(`piratehunter-backend`), поменять три env — код backend'а не трогаем

(claims mapping: `sub` → `fsp_id`, standard OIDC). Реестр достижений

заменяется боевым API с тем же контрактом ответа (200/404).

7. API

API PirateHunter

Открытый REST API платформы — `backend /api/v1` (JWT). Спецификация —

OpenAPI (Swagger UI на `/docs` при `BACK_ENV=local/staging`, в проде

закрыт). Канонические контракты версионированы (`/api/v1`), auth-слой

готов к публичному API с ключами (roadmap в конце). Internal API

core-api (`/internal/*`) и fsp_registry (`/fsp/v1`, `/oidc/*`) наружу не

выставлены (PRD §4.2) — с ними общается только backend.

Аутентификация

Три провайдера через интерфейс `IdentityProvider`, JWT HS256

(access короткий + refresh в httpOnly-cookie `ph_refresh`):

Метод	Путь	Описание
POST	`/auth/email/request`	код на email (SMTP; без SMTP — в лог, `dev_code` при `BACK_ENV=local`). TTL 600 с, кулдаун 60 с, 5 попыток, лимиты 10/сут на адрес и 30/сут с IP
POST	`/auth/email/verify`	`{email, code}` → JWT-пара (роль выбирается при первом входе)

Метод	Путь	Описание
POST	/auth/refresh	refresh по cookie новая access
POST	/auth/logout	сброс cookie
GET	/auth/fsp/authorize	вход через FSP ID: {authorize_url} мок-OIDC (login_hint=fsp_id)
GET/POST	/auth/fsp/callback	OIDC-callback: GET — browser (на origin фронта), POST — для API-клиентов
POST	/auth/dev	dev-login демо-аккаунтами (candidate / candidate-fsp / employer), только BACK_AUTH_DEV_LOGIN=1
GET	/users/me	профиль сессии (включая fsp_id)
POST	/users/me/role	выбор роли при онбординге (candidate / employer)

Чекбоксы при регистрации: согласие на обработку ПДн (152-ФЗ) и

отдельно на публикацию профиля.

Кандидат

Метод	Путь	Описание
GET/PUT	/candidate/profile	профиль: ФИО, контакты, стек, специализация, заявленный грейд, роли, стаж, софт-скиллы, согласия
POST	/candidate/profile/resume	загрузка PDF разбор в черновик профиля (текст обезличивается перед LLM; обязателен ручной review)
GET	/candidate/profile/pdf	стандартизированный PDF-профиль в гамме ФСП
POST/DELETE	/candidate/fsp-link	привязка/отвязка ФСП ID достижения + fsp_score + снапшот (404 = нет истории score 0)
GET	/candidate/grade	статус грейда, история тестов, кулдауны (смена грейда 90 дн, повтор теста 30 дн — даты доступности)
POST	/tests/sessions	старт CAT-теста `{grade_target, mode: full\	express}` первая

Метод	Путь	Описание	
			задача
POST	`/tests/sessions/{id}/answers`	ответ следующая задача \	результат `{theta, se, outcome, grade_verified, breakdown}`
GET	`/candidate/invitations`	входящие приглашения (компания, текст, **вилка ЗП**, способ связи)	
POST	`/candidate/invitations/{id}/accept`	принять контакты раскрываются обеим сторонам	
POST	`/candidate/invitations/{id}/decline`	отклонить	
GET	`/vacancies?filters`	опубликованные вакансии (опц. фича)	
POST	`/vacancies/{id}/apply`	отклик с сопроводительным	

Работодатель

Метод	Путь	Описание
GET/ PUT	`/employer/company`	профиль компании
CRUD	`/employer/needs`	потребности (raw_text + структура: spec, grade, stack_must/nice, вилка)
POST	`/employer/needs/parse`	LLM-разбор свободного текста «мы поняли так» (fallback — keyword)
GET	`/employer/needs/{id}/shortlist`	подборка: ranked кандидаты + explanation + блок «рядом с запросом»
GET	`/employer/market/salary?spec&grade`	виджет «вилка vs рынок»: p25/p50/p75 по данным платформы + размер категории
GET	`/employer/candidates?spec&grade&stack&fsp&category`	банк кандидатов с фильтрами
POST/	`/employer/invitations`	приглашения (вилка обязательна — 422 без неё; лимит

Метод	Путь	Описание
GET		100/час на компанию), трекер статусов
CRUD	`/employer/vacancies`	вакансии (вилка обязательна, gross/net подписан)
GET	`/employer/vacancies/{id}/applications`	входящие отклики, смена статусов

Справочник и общее

Метод	Путь	Описание
GET	`/catalog`	7 специализаций × 4 грейда (backend кэширует ответ core, TTL 300 с)
GET	`/health`	healthcheck

Формат ошибок

Единый стиль: HTTP-код + JSON `{ "detail": "человекочитаемое описание" }`.

- **422** — валидация схемы (Pydantic, тело с деталями полей) или

доменная валидация (нет вилки, неизвестный slug специализации);

- **401/403** — нет токена / чужой ресурс или роль не подходит

(зависимости FastAPI: кандидат не видит чужие приглашения,

работодатель не видит контакты до accept);

- **404** — ресурс не найден (включая «нет истории ФСП» у реестра);
- **409** — конфликт состояния: смена грейда в кулдауне, ФСП ID уже у

другого аккаунта;

- **429** — лимиты (коды на email, приглашения);
- **502** — core-api недоступен;
- **500** — `{ "detail": "внутренняя ошибка: <Type>: <msg>" }` (глобальный

handler, без стека наружу).

ATS-сценарий (roadmap-концепция Т3)

По определению из ответов 07.10: ATS = «синхронизация вакансий,

откликов и статусов с внутренней системой найма работодателя через API, чтобы платформа встраивалась в существующий процесс, а не требовала ручного дублирования». Концепция (в MVP не реализуем, схема зафиксирована):

1. Работодатель заводит API-ключ в кабинете; аутентификация `X-API-Key` (argon2-хэш ключа, паттерн известен).
1. Синхронизация вакансий: `POST /api/v1/employer/vacancies` из ATS (или полный обмен по webhook-эндпоинтам ATS).
1. Статусы: webhooks платформы по смене статусов приглашений/откликов (`sent → viewed → accepted/declined`) + маппинг на типовые этапы ATS-воронки (screening → interview → offer).

Roadmap API

- **Публичный API с ключами** (`/public/v1`, паттерн piratepitch): версионирование `/api/v1` и auth-слой уже готовы — дёшево.
- **MCP-сервер для ИИ-агентов**: инструменты «описать потребность → получить shortlist → отправить приглашение» поверх того же REST API.